

Final Project Report

AICS05 - VoyagerAI

Team Members:

SN	Student Name	Student ID
1.	Anirudhra Budhathoki	500236947
2.	Kabir Jatinkumar Pithadiya	500237694
3.	Nischal Pradhan	500237180
4.	Preet Soni	500237052
5.	Sahil .	500237277
6.	Sujan Thapa	500236989

Table of Content

1. Introduction.....	
2. Problem Statement.....	
3. System overview.....	
4. Scope/Analysis.....	
5. Frontend Development.....	
6. Backend API Engineering.....	
7. LLM Itinerary Engine.....	
8. Challenges Encountered.....	
9. Future Enhancement.....	
10. Method Used.....	
11. Conclusion.....	

Introduction

VoyagerAI is an intelligent travel-planning platform designed to simplify how users discover events, compare hotels and flights, and build personalized itineraries. Instead of navigating multiple websites and manually organizing travel details, VoyagerAI combines data from various sources into one unified system. Using AI-powered itinerary generation, semantic search, and real-time filtering, the platform creates optimized travel plans that match user preferences, budgets, and time constraints. By integrating modern technologies such as Supabase/PostgreSQL, FastAPI, Next.js, and large language models, VoyagerAI provides a fast, accurate, and user-friendly solution for planning trips more efficiently.

Problem Statement

Travel planning today is highly fragmented, requiring users to switch between multiple platforms to find events, compare hotels, check flight availability, map routes, and build itineraries. This scattered workflow leads to inefficiency, inconsistent information, and difficulty aligning schedules, budgets, and personal preferences. Existing systems lack integrated semantic search, real-time data aggregation, and automated itinerary generation that adapts to user context. The absence of a unified, intelligent platform creates significant friction for both individual and group travelers. VoyagerAI addresses this gap by consolidating multi-source travel data, applying AI-driven reasoning, and generating optimized, personalized itineraries through a scalable full-stack architecture.

System Overview

VoyagerAI is designed as a modular, full-stack travel intelligence platform that integrates data aggregation, AI-driven reasoning, and user-focused trip planning. The system collects events, hotels, flights, and points-of-interest (POI) data through multi-source scraping and API integrations, which are then cleaned, normalized, and stored in a structured Supabase/PostgreSQL database. A FastAPI backend manages all business logic, authentication, semantic search operations, and communication with the LLM-powered itinerary engine. The frontend, built with Next.js and TailwindCSS, provides an interactive interface for browsing events, adjusting filters, visualizing routes, and editing generated itineraries. Additional components—including Google Maps routing, distance calculations, deduplication pipelines, and group-planning features—work together to deliver personalized, optimized itineraries. The architecture is fully containerized with Docker and deployed on a cloud VM, ensuring scalability, maintainability, and efficient system performance.

Scope/Analysis

- Integration of multi-source data (events, hotels, flights, POIs) through APIs and web scraping.
- Data cleaning, normalization, deduplication, and storage in Supabase/PostgreSQL.
- AI-powered itinerary generation using LLMs with rule-based fallback logic.
- Semantic search and filtered search capabilities across all travel datasets.
- FastAPI backend with authentication, routing, and business logic for itinerary workflows.

- Next.js/TailwindCSS frontend for interactive travel planning and real-time itinerary editing.
- Google Maps routing, distance matrix calculations, travel-time estimation, and map visualization.
- Group itinerary planning with shared access, collaboration features, and preference matching.
- Budget planning and cost estimation for multi-day trips.
- Deployment using Docker containers on a cloud-based virtual machine.
- Code Control using SonarQube for code maintainability and performance metrics.
- End-to-end testing of APIs, UI components, and LLM itinerary logic.

Frontend Development

The frontend of VoyagerAI was engineered to provide a responsive, intuitive, and highly interactive travel-planning experience. Built using Next.js, the interface leverages server-side rendering and dynamic routing to ensure fast loading times and smooth transitions across pages. The UI is styled using TailwindCSS, allowing for a component-driven design system that maintains visual consistency while accelerating development.

A major focus of the frontend design was real-time itinerary interaction. Users can modify travel days, reorder activities, apply filters, and view hotel or event details in collapsible and modal components without page reloads. State management relies on React hooks and lightweight global stores, enabling seamless synchronization of itinerary changes with backend APIs.

Additionally, the interface incorporates map-based interactions using Google Maps APIs. Users can visualize travel routes, inspect distances between locations, and verify whether an itinerary is

realistically spaced within available travel time. Accessibility considerations were also integrated, including optimized contrast, keyboard navigation, logical tab ordering, and responsive layouts that support desktop, tablet, and mobile devices.

The frontend achieves a balance between aesthetics and performance. Through image optimization, code splitting, and caching strategies, the application remains smooth even when handling large datasets such as event listings or hotel comparisons. This ensures that VoyagerAI delivers a premium and modern user experience suitable for real-world travel planning.

Backend API Engineering

VoyagerAI's backend was developed using FastAPI, chosen for its asynchronous capabilities, speed, and lightweight design. The backend serves as the core engine of the system, managing communication between the frontend, database, AI models, and third-party services. It exposes more than 50 specialized endpoints for handling user authentication, event retrieval, hotel filtering, itinerary generation, and group planning workflows.

The API architecture follows modular principles, with dedicated service layers for events, hotels, flights, routes, LLM interactions, and user data. This separation of concerns ensures maintainability and scalability as new features are introduced. Database operations are performed using an ORM layer that cleanly maps PostgreSQL tables into Python objects, allowing efficient CRUD operations while reducing query complexity.

FastAPI's async execution model enables the backend to perform parallel data fetching, which is critical when retrieving event or hotel information from multiple external APIs simultaneously.

To avoid redundant calls and reduce latency, the backend implements intelligent caching strategies and rate-limit monitoring.

Security was a top priority in API development. JWT-based authentication protects user accounts, while middleware filters validate input, sanitize parameters, and enforce authorization rules. Error-handling modules ensure graceful responses for API failures, expired tokens, or unreliable third-party services. Logging and analytics allow monitoring of response times, failure points, and system load, helping maintain performance during peak usage.

Overall, the backend acts as a high-performance orchestration layer that integrates complex algorithms, external data sources, and user interactions into one cohesive system.

LLM Itinerary Engine

The itinerary engine is the intelligence backbone of VoyagerAI. It combines large language model (LLM) reasoning with deterministic rule-based logic to generate itineraries that are both creative and schedule-accurate. The LLM is responsible for interpreting user preferences, understanding context, ranking activities, and formulating coherent multi-day travel plans. However, instead of relying solely on AI, VoyagerAI enhances this process with custom constraints to ensure practical and conflict-free itineraries.

First, the engine preprocesses all available travel data—events, hotels, POIs, travel times, and budgets. It then constructs a structured representation of possible daily schedules. The LLM receives a detailed prompt containing trip duration, city characteristics, time windows, distances,

ticket prices, and user interests. It outputs a proposed itinerary, which is then validated by the rule layer.

The deterministic layer checks for issues such as overlapping activity times, unreachable locations, or budgets exceeding thresholds. If inconsistencies are found, the system automatically revises the itinerary by adjusting sequence order, replacing activities, or reallocating travel blocks. This hybrid design ensures that every proposed itinerary is both personalized and logically feasible.

The engine also supports group travel, where multiple user profiles are combined. Weighting algorithms measure compatibility between preferences and suggest activities that maximize group satisfaction. This makes VoyagerAI suitable for families, friends, or corporate travel planning. The result is a reliable, data-driven itinerary that adapts to user needs while maintaining real-world accuracy.

Database Engineering and Schema

VoyagerAI uses a Supabase/PostgreSQL relational database designed for efficiency, data integrity, and high-volume read/write operations. The schema is normalized to ensure that events, hotels, flights, users, itineraries, and search histories are stored without redundancy. Foreign keys enforce relationships between tables, such as linking itineraries to users or associating POIs with specific cities.

Indexes were applied to frequently queried columns—such as date ranges, city IDs, event names, and activity categories—to enable fast filtering and search operations. Additionally, text-search

capabilities were enhanced using PostgreSQL's vector embedding support, allowing semantic search queries that match user intent more closely than traditional keyword searches.

A dedicated deduplication pipeline ensures that incoming data from multiple APIs does not create duplicate entries. This involves fuzzy string matching, geospatial distance checks, and timestamp comparison to identify whether two listings represent the same event or venue. The system stores cleaned data in curated tables to reduce noise and improve AI accuracy.

To support itinerary storage, the database includes flexible JSONB fields that capture daily activity sequences, metadata, and user modifications. This hybrid relational–document approach combines structure with flexibility, enabling iterative trip editing without schema limitations.

Backup strategies and replication options were configured to ensure high availability and disaster recovery. Overall, the database architecture provides a solid foundation for scaling VoyagerAI to thousands of users and millions of records.

Challenges Encountered

Throughout the development of VoyagerAI, several technical and operational challenges emerged. One of the earliest issues was inconsistent data formats across APIs and scraped sources, requiring sophisticated normalization routines to standardize event names, timestamps, addresses, and pricing information.

Another challenge involved balancing LLM creativity with real-world feasibility. AI models often generate innovative but impractical itineraries. This required the team to build custom rule layers, travel-time validators, and conflict-resolution algorithms to maintain itinerary accuracy.

On the frontend, rendering large datasets without degrading performance required careful optimization. Virtual scrolling, API pagination, and memoized state components helped reduce lag. On the backend, managing API rate limits and preventing timeouts forced the team to introduce parallelization, caching, and retry mechanisms. Deployment challenges also surfaced, including Docker networking conflicts, environment variable handling, and VM resource constraints. Debugging these issues improved the team's understanding of production-level deployments.

Despite these obstacles, each challenge contributed to architectural refinements and stronger system reliability.

Future Enhancements

VoyagerAI's architecture has been intentionally designed to support continuous evolution, allowing new features to integrate seamlessly without disrupting existing workflows. One major enhancement planned is the introduction of real-time travel intelligence, where fluctuations in weather, event cancellations, transportation delays, and dynamic pricing are automatically incorporated into the itinerary. This would require integrating external live-data feeds and building predictive models capable of adjusting trip recommendations instantly.

Another key enhancement is the development of a personalized recommendation engine that learns user behavior over time. By analyzing historical trips, browsing behavior, preferred categories, spending tendencies, and travel pacing, VoyagerAI can generate highly individualized trip suggestions that feel intuitive and relevant. This system could also support group travel

preference merging, where conflicting priorities among members are resolved through weighted scoring algorithms.

The system's scope will also expand toward enabling in-platform booking, allowing users to reserve flights, hotels, and experiences directly through VoyagerAI. Implementing a secure, compliant payment infrastructure would transform the platform from a planning assistant into a fully integrated travel ecosystem. Finally, future releases may include a dedicated mobile application, offline itinerary access, and voice-based travel guidance, ensuring that users can rely on VoyagerAI throughout their entire travel journey.

User Experience and Interface Evaluation

The user experience (UX) of VoyagerAI was shaped around principles of clarity, continuity, and reduced cognitive load. The interface adopts a modular card-based layout that organizes complex travel information into digestible sections, making it easier for users to visualize their schedules and make quick adjustments. Interactive elements such as drag-and-drop itinerary editing, expandable detail views, and real-time search filtering enhance engagement and streamline the planning workflow.

Throughout development, usability testing identified several pain points that influenced iterative refinement. Early prototypes suffered from dense layouts that overwhelmed new users, prompting a transition toward cleaner spacing, intuitive navigation, and progressive disclosure of more advanced features. Visual hierarchy and color-coding were introduced to distinguish

between events, travel blocks, hotels, and POIs, enabling users to interpret itinerary structure at a glance.

Accessibility was also a central concern. The platform ensures compatibility with screen readers, keyboard-only navigation, and responsive scaling across screen sizes. Clear typography, consistent iconography, and frictionless map interactions further elevate the overall experience. These UX choices collectively ensure that VoyagerAI remains intuitive, visually organized, and enjoyable for both casual and power users.

Performance Optimization Strategies

Performance optimization in VoyagerAI was approached from both the frontend and backend to ensure responsiveness even under heavy data loads. On the backend, computational efficiency was improved through asynchronous execution, parallel API calls, and batched database operations. Intelligent caching layers were introduced to prevent redundant queries, significantly reducing latency when users repeatedly request similar data.

Frontend performance was enhanced using techniques such as code splitting, lazy loading, and memoization of expensive computations. Large datasets—like city-wide event listings—are rendered using virtualized lists to avoid unnecessary DOM updates. Map components and geospatial computations are offloaded to lightweight workers, ensuring smooth interface responsiveness.

Database optimization also played a crucial role. Proper indexing strategies, query normalization, and optimized join conditions dramatically improved retrieval times. To support scalability, the

system incorporates rate-limiting, load-balancing, and monitoring tools that automatically detect bottlenecks. Collectively, these techniques ensure VoyagerAI operates efficiently, even as traffic increases or data grows exponentially.

Algorithmic Decision Logic

VoyagerAI employs a hybrid decision-making framework that blends heuristic scoring, constraint satisfaction, and LLM-driven reasoning. The itinerary engine begins by evaluating candidate activities using a multi-factor scoring algorithm that considers relevance, proximity, popularity, cost, and user-specific preferences. Activities that score highly are promoted during itinerary assembly, ensuring alignment with the user's interests.

Next, the system applies a series of temporal and spatial constraints. Travel time estimators determine whether events can be realistically attended based on location, duration, and city traffic patterns. Overlapping activities are resolved using conflict-detection rules, while budget constraints trigger adaptive substitutions that reduce trip cost without sacrificing quality.

The LLM component provides high-level structuring and narrative coherence, generating recommendations that feel personalized and natural. However, the deterministic logic ensures the final result is grounded in real-world feasibility. This hybrid architecture enables VoyagerAI to create itineraries that are both adaptive and reliable, balancing creativity with precision.

Security Considerations

Security is foundational to VoyagerAI’s architecture, given the sensitivity of user profiles, travel histories, and authentication data. The backend implements JWT-based authentication to safeguard session integrity, while passwords are hashed using industry-standard algorithms before storage. All service communication occurs over encrypted HTTPS channels to prevent interception of sensitive information.

Supabase’s Row Level Security (RLS) ensures strict access control, restricting database visibility exclusively to the authenticated user’s records. Input validation and sanitation prevent injection attacks, while role-based permissions ensure administrative actions cannot be executed by standard users. Additionally, rate-limiting protects APIs from brute-force attempts or excessive traffic spikes. Continuous security auditing is performed via automated dependency checks and SonarQube analysis, helping identify vulnerabilities such as outdated libraries or code smells. Logging, monitoring, and anomaly detection ensure rapid response to suspicious behavior. These measures collectively establish a secure ecosystem capable of protecting user trust as the platform scales.

Testing Methodology

Testing VoyagerAI required a comprehensive, multi-layered approach to validate correctness, reliability, and user experience across the entire system. Unit tests ensured that isolated components—like API endpoints, data-cleaning functions, and scoring algorithms—behaved as

expected under both typical and edge-case conditions. Integration tests validated interactions between the backend, database, and external APIs, ensuring data pipelines operated smoothly.

Frontend testing leveraged tools such as React Testing Library to simulate real user actions, verifying that UI components rendered correctly, forms validated input properly, and itinerary editing behaved consistently. Cross-browser compatibility testing ensured standardized behavior across Chrome, Firefox, and mobile browsers.

User Acceptance Testing (UAT) played a critical role by evaluating real-world scenarios, such as planning multi-day trips, adjusting budgets, and switching cities. Feedback gathered during UAT informed UX improvements and helped refine confusing workflows. Performance and load testing further validated system behavior under heavy traffic. This layered approach ensured that VoyagerAI delivered a robust and polished user experience.

Quality Assurance Practices

Quality assurance for VoyagerAI was grounded in continuous integration, code quality monitoring, and structured review practices. SonarQube was integrated into the development pipeline to track maintainability, complexity, and security vulnerabilities. This allowed developers to identify problematic code early and enforce clean coding standards.

Code reviews became a routine part of the workflow, promoting knowledge sharing and ensuring adherence to architectural guidelines. Automated linting, formatting tools, and typed interfaces reduced inconsistencies and prevented bugs caused by loosely structured logic.

On the process side, sprint retrospectives and documentation updates ensured that the team continuously improved development efficiency. Version control best practices, feature branching, and merge conflict resolution helped maintain project stability. The resulting codebase is modular, maintainable, and prepared for long-term scalability.

Scalability and Deployment Strategy

VoyagerAI's deployment strategy emphasizes reliability, portability, and horizontal scalability. Docker containers ensure consistent runtime environments and simplify deployment across development, staging, and production servers. A reverse proxy manages SSL, routing, and fault isolation, enabling secure and efficient request handling.

The platform is engineered to scale horizontally by deploying additional backend instances under increasing load. Database performance scaling is supported through indexing, sharding options, and read replicas if required. Caching layers reduce redundant computation and improve overall system throughput.

Monitoring tools capture performance metrics, memory usage, and error logs, providing actionable insights when optimizing system behavior. Auto-restart mechanisms and health checks ensure continuous uptime. This architecture allows VoyagerAI to grow steadily while maintaining strong performance under real-world conditions.

Data Cleaning and Deduplication Deep Dive

Data cleaning was one of the most critical components in building a reliable travel recommendation system. Since sources vary widely in formatting, language, and labeling, VoyagerAI employs sophisticated normalization pipelines to reconcile differences in event titles, venue locations, descriptions, categories, and dates. Missing information is inferred when possible using heuristics and geospatial reasoning.

The deduplication system uses fuzzy matching techniques—such as cosine similarity and Jaro-Winkler distance—to detect near-identical records that originate from multiple providers. Geographic clustering ensures that events occurring at the same coordinates aren't duplicated, while timestamp comparison distinguishes between recurring events and one-time listings.

To prevent propagating errors into the recommendation engine, cleaned data is validated and stored in curated tables. This rigorous data hygiene process ensures that users receive accurate, consistent, and trustworthy travel information.

Semantic Search and Ranking Mechanics

VoyagerAI's search engine is powered by vector embeddings that interpret meaning rather than relying on simple keywords. Queries are transformed into semantic vectors and compared against a database of event and POI embeddings, enabling intelligent matching even when users use vague or unconventional phrasing.

Ranking extends beyond relevance. The system weights factors such as distance, popularity, category alignment, schedule compatibility, and user-specific profile preferences. These weights are dynamically adjusted based on feedback loops and performance analysis.

As the dataset grows and more user interactions are logged, the ranking engine becomes increasingly accurate, continually refining its understanding of user behavior and delivering recommendations tailored to evolving preferences.

Methods Used

Data Collection & Processing Methods

- Used API integrations (Ticketmaster, Eventbrite, Skyscanner, Booking.com) to fetch structured travel data.
- Developed Python-based web scrapers for sites without APIs.
- Implemented data cleaning pipelines to remove duplicates, normalize fields, and standardize formats.
- Used Supabase/PostgreSQL for structured storage, indexing, and relational mapping.

Backend Development Methods

- Built REST APIs using FastAPI, enabling fast, asynchronous back-end operations.
- Implemented modular service architecture for events, hotels, flights, and itinerary generation.
- Integrated authentication and session management for user access and group planning features.
- Applied semantic search techniques using vector-based embeddings and filtering logic.

AI & Itinerary Generation Methods

- Used LLM-based prompt engineering to generate travel itineraries dynamically.
- Created fallback rule-based algorithms to handle timing conflicts, travel durations, and budgets.
- Integrated Google Maps Distance Matrix API for travel-time calculations.
- Applied data-driven heuristics for ranking events, POIs, and hotels.

Frontend & UI/UX Development Methods

- Developed the frontend with Next.js + TailwindCSS for speed and responsiveness.
- Used state management for real-time itinerary updates.
- Implemented dynamic filtering, sorting, and map-based interaction for improved usability.

DevOps & Deployment Methods

- Containerized the entire system using Docker for consistent deployment.
- Deployed backend and frontend on a cloud virtual machine with environment isolation.
- Applied CI/CD workflows for continuous integration and automated updates.

Code Quality Control & Testing Methods

- Used SonarQube for code quality assessment, ensuring low technical debt and high maintainability.
- Performed API testing using Postman and automated scripts.
- Conducted end-to-end user testing, validating itinerary workflow, navigation, and UI reliability.

GIT Repository:

The screenshot shows the GitHub repository page for 'VoyagerAI' (Public). The repository is managed by 'kabirpithadiya' and has 12 branches and 0 tags. The main branch is 'dev'. The repository contains several files and folders, including 'backend', 'frontend', '.env.docker', '.gitignore', 'docker-compose.yml', 'dockerfile', 'env.example', 'package-lock.json', 'package.json', 'setup.sh', 'setup_database.sh', 'setup_postgres.sh', and 'start_all.sh'. The repository has 147 commits and 1 star. The right sidebar shows the 'About' section with a description, 'Activity' section with 1 star, 0 watching, and 1 fork, and 'Releases' section with no releases published. The 'Packages' section also shows no packages published. The 'Contributors' section shows 7 contributors.

Sonarcube:

The screenshot shows the SonarQube dashboard for the 'VoyagerAI' project. The dashboard displays the 'Main Branch Summary' with 15k Lines of Code and a last analysis 30 seconds ago. The Quality Gate is 'Passed'. The dashboard also shows 'New Code' and 'Overall Code' sections. The 'New Code' section shows 26 New Issues, 0 Accepted Issues, and 2.36% Duplications. The 'Overall Code' section shows 0 Security Hotspots. The dashboard also includes a 'Coverage' section with a link to 'Set up coverage analysis'.

AIP-F25-2 > VoyagerAI > dev

Last analysis had a warning

Summary
Issues
Security Hotspots
More

	Lines of Code	Security	Reliability	Maintainability	Security Hotspots	Coverage	Duplications
VoyagerAI							
backend	6,902	0	0	0	0	—	1.8%
frontend	8,029	0	23	84	0	—	0.1%
package-lock.json	—	0	0	0	0	—	0.0%
package.json	—	0	0	0	0	—	0.0%
setup.sh	—	0	0	0	0	—	0.0%
setup_database.sh	—	0	0	0	0	—	0.0%
setup_postgres.sh	—	0	0	0	0	—	0.0%
start_all.sh	—	0	0	0	0	—	0.0%
start_backend.sh	—	0	0	0	0	—	0.0%
start_frontend.sh	—	0	0	0	0	—	0.0%
test_auth.html	107	0	0	1	0	—	0.0%

Conclusion

VoyagerAI demonstrates a powerful and integrated approach to modern event and travel planning using advanced search, optimization, and user-centered design. By combining multi-source event ingestion with semantic ranking and intelligent itinerary optimization, the system addresses shortcomings in existing tools while establishing a strong foundation for future capabilities such as full travel booking integration and enhanced personalization via machine learning. The project reflects robust teamwork, strong research application, and a technically sound implementation aligned with professional and academic expectations.

References

- Ticketmaster. (2025). *Discovery API v2 documentation*. Ticketmaster Developer Portal.
- Skyscanner Ltd. (2024). *Skyscanner Travel APIs: Technical documentation*.
<https://developers.skyscanner.net>
- OpenStreetMap Foundation. (2025). *OpenStreetMap database and geospatial metadata*.
<https://www.openstreetmap.org>
- Supabase. (2024). *Supabase documentation: Postgres backend, auth, and storage*.
<https://supabase.com/docs>
- Supabase. (2023). *Supabase architecture and open-source backend overview*.
<https://supabase.com>
- PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*.
<https://www.postgresql.org/docs>
- Scrapy Developers. (2024). *Scrapy web-scraping framework documentation*.
<https://docs.scrapy.org>
- Eventbrite. (2024). *Eventbrite API v3: Technical reference*.
<https://www.eventbrite.com/platform/api>
- Amadeus IT Group. (2024). *Flight & hotel search APIs: Developer documentation*.
<https://developers.amadeus.com>
- Aouad, A., Zhang, Z., & Eberle, W. (2021). *An intelligent itinerary planning system for personalized trip recommendations*. *Expert Systems with Applications*, 184, 115521.
<https://doi.org/10.1016/j.eswa.2021.115521>
- Microsoft. (2024). *GitHub Actions for Azure deployments*. GitHub Documentation.
<https://learn.microsoft.com/azure/developer/github/actions>

- Pahl, C. (2015). Containerization and the PaaS cloud. *IEEE Cloud Computing*, 2(3), 24–31. <https://doi.org/10.1109/MCC.2015.51>
- SonarSource S.A. (2024). SonarQube: Clean code and static analysis platform. <https://www.sonarsource.com/products/sonarqube/>
- Crummy, K. (2024). Beautiful Soup documentation. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Grinberg, M. (2018). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media.
- Ferreira, M., & Supabase Engineering Team. (2022). Supabase: Building scalable open-source backend systems on Postgres. *Proceedings of the Open Source Summit*, 112–125.
- OpenAI. (2024). OpenAI API documentation: Chat completions, embeddings, and assistants APIs. <https://platform.openai.com/docs>
- OpenAI. (2022). ChatGPT: Optimizing language models for dialogue. <https://openai.com/blog/chatgpt>
- SonarQube:
https://sonarcloud.io/summary/overall?id=AIP-F25-2_VoyagerAI&branch=dev

Sign off:

Anirudhra Budhathoki

Kabir Jatinkumar Pithadiya

Nischal Pradhan

Preet Soni

Sahil

Sujan Thapa

Submitted for:

AICS05 – VoyagerAI

Loyalist College Toronto Campus

Date:

2025-12-04